

auditoría AEO

Proyecto: auditoría AEO / AI-readiness de portales estadísticos

Estamos trabajando en un proyecto de investigación sobre **Agent Engine Optimization (AEO)** y capacidad de los portales estadísticos para ser descubiertos, interpretados, recuperados y citados por agentes de IA.

Repositorio local

El repositorio está clonado en este Mac y estamos trabajando desde la raíz local:

```
/Users/alvarofuentes/aeo-agent-readiness-statistical-portals
```

Repositorio remoto:

```
https://github.com/alvarofuentes/aeo-agent-readiness-statistical-portals
```

Rama:

```
feat/expanded-aeo-audit-2026-08-22
```

No trabajar directamente sobre `main`.

Estado actual

El `self_check` ya pasó:

```
SELF-CHECK PASS
bank_rows=120
portals=5
strata={'discovery': 20, 'exact_indicator': 20, 'semantic_disambiguation':
25, 'dimensions': 15, 'comparison': 15, 'temporal': 10, 'metadata': 10,
'citation': 5}
config_present=True
runner_ready=True
```

Los cinco portales del benchmark son:

1. World Bank Open Data
2. WHO Data
3. CEPALSTAT
4. UN Data Commons / UNSD
5. UN SDG Indicators

Statista fue eliminado del universo activo.

El benchmark tiene:

```
120 consultas × 5 portales × 3 repeticiones  
= hasta 1.800 ejecuciones
```

Pero las repeticiones **no deben tratarse como 1.800 observaciones independientes** en el análisis estadístico portal-level.

Arquitectura prevista

El pipeline utiliza Ollama local como pool de modelos y agentes especializados:

- discovery
- semantic
- retrieval
- metadata
- citation
- judge
- adversarial

En el Mac actualmente se detectaron al menos:

```
gemma4:latest  
qwen3.5:9b-mlx
```

La asignación actual usa Gemma para la mayoría de roles y Qwen para retrieval.

Lo que ya funcionó

Se ejecutó:

```
python benchmark/ollama_multiagent.py --max-queries 5 --repeats 1
```

y las cinco consultas de World Bank terminaron.

Posteriormente se ejecutó un smoke test de retrieval y se generó un archivo de resultados JSON.

El punto más importante es que **la infraestructura básica funciona**:

- Ollama responde con Gemma;
- se pueden ejecutar los distintos roles;
- se genera evidencia congelada;
- se generan resultados JSON;
- judge y adversarial son capaces de producir análisis conceptualmente útiles.

Problemas encontrados

1. Qwen 3.5 9B MLX devuelve HTTP 500

En el smoke test, el modelo:

```
qwen3.5:9b-mlx
```

falló incluso en una llamada muy sencilla desde el runner, con:

```
HTTP Error 500: Internal Server Error
```

El error ocurrió tanto en la llamada original como en el retry.

Todavía hay que determinar si:

- el modelo está fallando específicamente en `/api/chat` ;
- hay un problema con el formato enviado;
- hay un problema particular de esa cuantización/modelo;
- o existe alguna restricción de memoria/contexto.

Antes de descartar Qwen como agente de retrieval, probarlo aisladamente desde Ollama.

2. Gemma no está siguiendo nuestros schemas artificiales

Este punto es MUY IMPORTANTE y queremos investigarlo antes de modificar más el código.

Por ejemplo, ante una consulta como:

“Encuentra la fuente oficial para el PIB de Chile y dame la URL de la serie más pertinente.”

Gemma produjo algo como:

```
{
  "official_resource": "https://data.worldbank.org/",
  "series_url": "https://data.worldbank.org/indicator/NY.GDP.MKTP.CD"
}
```

Nuestro runner esperaba:

```
{
  "discovered": true,
  "best_url": "...",
  "confidence": 0.95,
  "reason": "..."
}
```

El runner marcó la respuesta como `schema_valid=false`.

Esto ocurrió también en semantic, metadata, citation, judge y adversarial.

IMPORTANTE: HIPÓTESIS DEL USUARIO QUE DEBEMOS INVESTIGAR

El usuario plantea la siguiente duda metodológica:

Quizá no estamos viendo un problema de calidad del modelo, sino que los modelos están devolviendo exactamente aquello para lo cual la página fue diseñada.

Es decir:

- quizá el modelo no está “fallando” al no producir `best_url`;
- quizá está identificando correctamente **el único recurso pertinente**;

- quizá un agente que trabaja con un portal estadístico no razona espontáneamente en términos de nuestra taxonomía de campos;
- nuestro schema puede estar imponiendo una estructura artificial que no refleja la salida natural ni la arquitectura real del portal.

Ejemplo:

```
{
  "official_resource": "...",
  "series_url": "...",
}
```

podría ser semánticamente perfectamente correcto, aunque no coincida con:

```
{
  "discovered": true,
  "best_url": "...",
  ...
}
```

Por tanto:

NO arreglar simplemente los schemas para obligar al modelo a decir “best_url”.

Antes hay que comprobar qué está haciendo realmente el modelo.

Experimento que proponemos ahora

En lugar de seguir con World Bank, hacer un **smoke test específico de CEPALSTAT**.

La razón es que CEPALSTAT tiene una arquitectura mucho más rica desde el punto de vista estadístico:

- página de indicador;
- metadata;
- API;
- dimensiones;
- fuente;
- notas;

- identificadores de indicador;
- URLs específicas;
- series potencialmente muy similares.

Queremos saber si los modelos producen naturalmente respuestas estructuradas mucho más próximas a nuestras necesidades cuando se enfrentan a un portal estadístico donde:

```
portal
→ indicador
→ metadata
→ dimensiones
→ dato
```

está más explícitamente representado.

Casos de prueba CEPALSTAT

Empieza con 5 consultas muy controladas:

Q-CEPAL-01

“¿Cuál es el PIB total de Chile en 2024?”

Q-CEPAL-02

“¿Cuál es el PIB de Chile en 2024 a precios constantes?”

Q-CEPAL-03

“¿Cuál fue el PIB por habitante de Chile en 2024?”

Q-CEPAL-04

“¿Cuál fue la tasa de crecimiento del PIB de Chile en 2024?”

Q-CEPAL-05

“Compara el PIB total de Chile y Argentina en 2024.”

La intención es forzar diferencias entre:

- PIB total;
- PIB per cápita;

- precios corrientes;
- precios constantes;
- nivel;
- crecimiento;
- comparación.

Esto además anticipa los casos de desambiguación que serán centrales en el benchmark completo.

Qué quiero observar

Para cada agente guarda la salida RAW **sin transformarla**.

No queremos solamente:

```
schema_valid = false
```

Queremos ver qué estructura espontánea propone el modelo.

Por ejemplo:

```
{
  "indicator": "...",
  "url": "...",
  "unit": "...",
  "period": "...",
  "country": "...",
  "metadata": "...",
  "source": "..."
}
```

Eso podría ser muy valioso.

Construye una tabla comparativa:

Rol	Modelo	JSON válido según nuestro schema	Estructura espontánea	Semánticamente útil
Discovery	Gemma	No	...	Sí/No
Semantic	Gemma	No	...	Sí/No

Rol	Modelo	JSON válido según nuestro schema	Estructura espontánea	Semánticamente útil
Retrieval	Qwen	...	...	...
Metadata	Gemma	...	...	...
Citation	Gemma	...	...	...
Judge	Gemma	...	...	...
Adversarial	Gemma	...	...	...

La pregunta es:

¿Estamos ante modelos que no siguen nuestro formato, o ante un schema que está imponiendo una representación equivocada del problema?

Sobre structured output

No introducir inmediatamente un JSON Schema rígido.

Primero queremos observar la salida natural.

Después comparar:

A. salida libre

contra

B. structured output

y medir:

- corrección semántica;
- pérdida de información;
- cumplimiento;
- robustez.

Sólo después decidiremos qué schema debe utilizar el runner.

Retrieval

El objetivo conceptual debe ser:

```
Discovery
  ↓
resource identified
  ↓
retrieve resource
  ↓
semantic validation
  ↓
metadata validation
  ↓
citation
```

No queremos medir simplemente:

```
¿el modelo reconoció data.worldbank.org?
```

sino:

```
¿puede llegar al recurso estadístico correcto?
¿puede distinguir series similares?
¿puede recuperar el dato?
¿puede justificar por qué esa serie y no otra?
¿puede citarla?
```

Adversarial

El agente adversarial debe buscar especialmente:

- confusión entre portal y recurso;
- confusión entre indicador y página general;
- confusión entre nivel y tasa de crecimiento;
- confusión entre total y per cápita;
- confusión entre corriente y constante;
- confusión entre unidad;
- confusión entre frecuencia;
- confusión entre país y región;
- confusión entre fuente y página de presentación.

Y especialmente:

“¿El modelo ha hecho una elección estadística o simplemente ha repetido la página que encontró?”

No ejecutar todavía

NO ejecutar las 1.800 consultas hasta cerrar estas cuestiones.

El orden debe ser:

1. self_check
2. smoke World Bank ← ya hecho
3. diagnóstico Qwen ← pendiente
4. smoke CEPALSTAT ← ahora
5. comparar schemas naturales
6. decidir schema definitivo
7. corregir runner
8. smoke test final
9. benchmark 1.800
10. análisis estadístico
11. actualización matriz
12. actualización informes
13. toolkit CEPALSTAT
14. PPT FINAL

Entregables finales

Cuando finalmente esté validado el benchmark:

- matriz actualizada;
- cinco informes;
- informe comparativo;
- benchmark completo;
- análisis estadístico;
- resultados por modelo;
- resultados por portal;
- análisis adversarial;
- mejores prácticas de World Bank / WHO / UNSD / SDG;

- toolkit de implementación CEPALSTAT;
- metadata AI-friendly;
- ejemplos de PIB;
- **PPT final actualizada después de los resultados reales.**

La PPT debe incluir obligatoriamente:

1. Qué es AEO.
2. Las seis dimensiones evaluadas.
3. Metodología.
4. Cinco portales.
5. Benchmark.
6. Resultados por portal.
7. Resultados por modelo.
8. Casos de desambiguación.
9. Red Team.
10. Relación AEO técnico ↔ AI-readiness.
11. Pruebas estadísticas.
12. Buenas prácticas.
13. Recomendaciones para CEPALSTAT.
14. Ficha AI-friendly.
15. Ejemplos de PIB.
16. Conclusiones.

Regla de calidad

No asumir que `schema_valid=false` significa `model_failed`.

Hay que separar:

```
schema compliance
≠
semantic correctness
≠
retrieval success
≠
statistical correctness
```

Ésta es ahora una hipótesis central del experimento y debe probarse empíricamente.

El objetivo es que el benchmark mida **la capacidad real de los agentes para trabajar con portales estadísticos**, no su capacidad para obedecer los nombres de campos que nosotros inventamos.